Telegram Serverless

Fonte: https://core.telegram.org/bots/serverless · Extraído em 11 de novembro de 2026

Telegram Serverless lets you run backend code for your bot and Mini App **directly on Telegram's infrastructure** — no servers to provision, no containers to keep alive, no scaling to think about. You write plain JavaScript modules, deploy them with a single command, and Telegram runs them in a fast, isolated V8 sandbox that sits right next to the Bot API and a built-in database.

If you have ever wired a bot to a VPS, a cloud function, or a hosting panel just to answer a `/start`, this is the part you no longer have to do.

On this page

- Why serverless
- Getting started
- Building with AI
- On the go with BotFather
- Projects & modules
- The database
- The SDK
- CLI reference

Recent changes

October 6, 2026

- Added Mini App hosting. The build output of your Mini App's front-end is deployed together with the bot and served at `https://app<app_id>.tgcloud.ai/`.
- Added endpoints, the functions your Mini App calls with Telegram.WebApp.Serverless.call.
- Updated the CLI to version 0.2.0.
- Added the project config file tgcloud.jsonc.
- Project modules can now be imported by relative path, with the `.js` extension.
- All modules now live in the `tgcloud/` folder. Run npx tgcloud upgrade once in a project created with an earlier version of the CLI.

Why serverless

A Telegram bot is, at heart, a program that reacts to updates. Traditionally you had to host that program somewhere that is always on, reachable, and secure — and then keep it that way. Telegram Serverless removes that layer entirely:

- **No infrastructure.** There is no machine to rent, patch, or monitor. Your code runs on demand and scales with your bot automatically.

- **Batteries included.** The Telegram Bot API, an SQLite-backed database, and outbound HTTP are available to every module out of the box — nothing to install, no credentials to wire up.
- **Fast, isolated execution.** Each invocation runs in a lightweight V8 isolate, close to Telegram's own systems, so calls to the Bot API and your database are quick and reliable.
- **A real developer workflow.** A project lives in a folder on your machine under version control. You edit files, see exactly what changed, deploy atomically, and roll your database schema forward with reviewed migrations — the way you already work with everything else.

The mental model

You work in **three places**, and they map cleanly onto each other:

Where	What lives there
Your project folder	JavaScript modules — schema, shared code, update handlers, Mini App endpoints — and, if you have one, the Mini App's front-end
The cloud	The deployed copy of those modules, your bot's database, and the Mini App's files
The `tgcloud` CLI	The bridge — it shows you differences and syncs them

You never SSH into anything. You edit files locally, run `npx tgcloud push`, and the platform takes it from there. Your bot's traffic is handled by the deployed modules; your database persists between invocations.

A project has just four kinds of code, all inside one `tgcloud/` folder:

```
tgcloud/
├─ endpoints/  # functions your Mini App calls
├─ handlers/   # functions that react to Telegram update types
├─ lib/        # shared code you import from anywhere
└─ schema.js   # your database tables
```

When an update arrives — a message, a button press, an inline query — Telegram routes it to the matching handler (`handlers/message.js` , `handlers/callback_query.js` , …) and calls its default export. That function talks to the Bot API and the database through the SDK, and returns. That is the whole loop. An update with no matching handler is simply ignored, so you add only the handlers you need.

A Mini App is optional. If your bot has one, its front-end lives in the same project, next to `tgcloud/` . Serverless hosts its build output and runs the `endpoints/` it calls. See Mini App front-end.

Quick demo

Here is a complete, working demo bot. It replies to every message and remembers how many it has seen from each chat.

`tgcloud/schema.js`

```
import { table, integer } from 'sdk/db';

export const counters = table('counters', {
  chatId: integer('chat_id').primaryKey(),
  seen:   integer('seen').notNull().default(0),
});
```

`tgcloud/handlers/message.js`

```
import { api, db } from 'sdk';
import { sql } from 'sdk/db';
import { counters } from '../schema.js';

export default async function (message) {
  const chatId = message.chat.id;

  // Insert the counter, or bump it if this chat already has one — and get the
  // resulting row back in the same statement via .returning().
  const [row] = await db.insert(counters)
    .values({ chatId, seen: 1 })
    .onConflictDoUpdate({
      target: counters.chatId,
      set: { seen: sql`${counters.seen} + 1` },
    })
    .returning()
    .run();

  await api.sendMessage({
    chat_id: chatId,
    text: `Hello! I've seen ${row.seen} message(s) from you.`,
  });
}
```

Deploy it:

```
npx tgcloud push       # upload the modules
npx tgcloud migrate    # create the `counters` table
```

That's a live bot with persistent state and no server. Everything in it — `api`, `db`, the `table()` DSL — is described in the sections below.

Serverless is a general backend for Telegram bots and Mini Apps, not a template for one kind of app. It is ideal for:

- **Conversational AI Bots** that need to store per-user state in a database.
- **Mini App Backends** that store user data and serve dynamic content.
- **Games and Tools** — including leaderboards, quizzes and more.
- **Automations and Integrations** that call third-party HTTP APIs and push results into chats.

Getting started

This walkthrough takes you from an empty folder to a live bot that answers messages and stores data. It assumes you have Node.js 18 or newer installed and a bot registered with @BotFather. By the end you will have used every command you need day to day: `push`, `migrate`, `run`, and `status`.

> **Before anything else, switch Serverless on.** In @BotFather, open your bot → **Serverless** and turn it on. That turns the feature on for this bot and unlocks its CLI access token, handlers, library, and database.

1. Create a project

The fastest way to start is the project creator, which scaffolds a project and installs the CLI into it:

```
npm create @tgcloud/bot example_bot
cd example_bot
```

The argument is the target folder: pass `.` to scaffold into the current folder, or any path. It works in an existing folder too and never overwrites files you already have.

This gives you a ready-to-edit project:

```
example_bot/
├─ tgcloud/             # everything the platform runs
│  ├─ handlers/
│  │  └─ message.js     # a starter message handler (echoes text back)
│  ├─ lib/              # your shared modules go here (empty to start)
│  └─ schema.js         # your database tables
├─ docs/
│  └─ tgcloud-sdk.md    # SDK reference (for you and your AI tools)
├─ AGENTS.md            # orientation for AI coding assistants
```

```
├─ package.json
└─ tgcloud.jsonc        # project config
```

The scaffolded files are self-documenting — each one contains commented examples of what you can do next.

The CLI installs into the project as a local dev-dependency, so you run it with `npx tgcloud <command>` (npx finds the copy in your project's `node_modules`), or through the `npm run` shortcuts the scaffold adds to `package.json` (`npm run deploy`, `npm run status`). By default there is no global `tgcloud` on your `PATH`.

You can also install it globally — `npm install -g @tgcloud/cli` — if you'd rather type a bare `tgcloud` from anywhere. That's handy for running `tgcloud init` in any empty folder, and it's what shell tab-completion needs. Either way you get the same project.

> **Building a Mini App?** Start from its front-end instead. Create it with your usual tool, then run the project creator in that folder. See Mini App front-end.

2. Link your bot

Every project is tied to one bot. Connect them with `login`, which asks for your CLI access token (@BotFather → your bot → Serverless → CLI Access → Access token — a separate token from your bot's API token) and stores it locally:

```
npx tgcloud login
```

The token has the form `app<id>:<secret>`. The CLI keeps it in `.tgcloud/`, which is git-ignored, and never prints the secret part. Login is the only time you are asked for it — see Authentication for how tokens are resolved in CI.

3. Look around

Two commands tell you where things stand at any moment, both fully offline:

```
npx tgcloud status     # what has changed locally vs. the deployed copy
npx tgcloud diff       # the line-by-line changes
```

Right after `init` everything is new and nothing is deployed yet. `status` shows the starter files waiting to go up.

4. Deploy

Send your modules to the cloud:

```
npx tgcloud push
```

`push` uploads every changed module in one atomic batch and updates your local record of what the cloud now holds. Your bot is live: open it in Telegram and send it a message — the starter handler echoes it back.

> **Deploying never touches your database.** Pushing code and changing your database schema are deliberately separate steps, so a code deploy can never surprise you with a data migration. That is what the next step is for.

5. Add a database table

Let's make the bot remember something. Open `tgcloud/schema.js` and declare a table:

```
import { table, integer, text, sql } from 'sdk/db';

export const messages = table('messages', {
  id:      integer('id').primaryKey({ autoIncrement: true }),
  chatId:  integer('chat_id').notNull(),
  text:    text('text'),
  created: integer('created_at', { mode: 'timestamp'
}).default(sql`(unixepoch())`),
});
```

Deploy the schema, then apply it to the database:

```
npx tgcloud push       # uploads the new schema
npx tgcloud migrate    # creates the `messages` table
```

`push` reports that the schema is out of sync and shows you the pending change, but applies nothing. `migrate` walks you through the change and, on your confirmation, creates the table. This two-step model — and what happens with riskier changes like drops — is covered in Migrations.

6. Store and read data

Now use the table from your handler. Edit `tgcloud/handlers/message.js`:

```
import { api, db } from 'sdk';
import { eq } from 'sdk/db';
import { messages } from '../schema.js';

export default async function (message) {
```

```
  // Save this message.
  await db.insert(messages)
    .values({ chatId: message.chat.id, text: message.text })
    .run();

  // Count how many we've stored for this chat.
  const count = await db.$count(messages, eq(messages.chatId,
message.chat.id));

  await api.sendMessage({
    chat_id: message.chat.id,
    text: `Saved. That's ${count} message(s) from this chat so far.`,
  });
}
```

Deploy the updated handler with `npx tgcloud push`, then send your bot a few messages and watch the count climb. The database persists between invocations — that's your bot's memory.

7. Test without deploying

You don't have to deploy to try a change. `npx tgcloud run` executes a handler on the platform using your **local** files, without publishing them:

```
npx tgcloud run handlers/message '{ chat: { id: 1 }, text: "hello" }'
```

The argument is the payload your handler receives — for `handlers/message`, a Message — written in JSON5 (so you can skip quoting keys). The command prints anything the handler logged with `console.*`, the return value, and how long it took. This is the tightest loop for iterating on logic — no deploy, no waiting for a real message.

8. Keep in sync

As you work, a handful of commands keep your local project and the cloud aligned: `npx tgcloud status` shows what changed, `npx tgcloud push` deploys, `npx tgcloud pull` brings your local project in line with the cloud, `npx tgcloud fetch` checks what is deployed without touching your files, and `npx tgcloud reset` discards local changes.

> If two people (or two machines) deploy to the same bot, the platform detects the conflict and `push` stops to let you `pull` first — you can never silently overwrite someone else's work. See Staying in sync.

Building with AI

Prefer to build with an AI assistant — or is the only coder on your team an AI? You can still ship a bot. We've taken a first step to make an AI agent feel at home in a project: every new one is

scaffolded with an `AGENTS.md` and a `docs/tgcloud-sdk.md` reference that agentic coding tools read automatically.

Together with a small, self-contained runtime — one SDK, no npm packages to wrangle — that gives the assistant a running start on the conventions generic codegen tends to miss here: relative imports with `.js`, no foreign keys, every `db` call is async, one handler per update type, and the two-step `push` / `migrate` flow.

Try it:

```
npm create @tgcloud/bot my-bot
cd my-bot
opencode            # or Claude Code, Cursor, … — any agent that reads
AGENTS.md
```

Then just ask, in plain language:

> Write a bot that remembers each person's to-do list — add an item when they send text, and show the whole list when they send /list.

The assistant edits `schema.js` and your handlers for you; you review, test a change instantly with `npx tgcloud run`, then go live with `npx tgcloud push` and `npx tgcloud migrate`. `AGENTS.md` is part of your project — edit it as the bot grows so the guidance stays accurate.

On the go with BotFather

Down to just your phone? The whole project lives in @BotFather too — open your bot → **Serverless** and you get everything the CLI manages, on a touchscreen:

- **Handlers** — create, edit, and test-run update handlers; BotFather keeps the webhook in sync with the handlers you have (the same *In sync* / *Out of sync* the CLI reports).
- **Endpoints** — the functions your Mini App calls.
- **Library** — your shared `lib/` modules.
- **Database** — edit `schema.js` in the same Drizzle-like syntax, review pending changes, and apply them; **Save** deploys.
- **Mini App Files** — the front-end your users open in Telegram.
- **CLI Access** — grab the CLI access token here when you're back at a keyboard.

It's one and the same cloud project, so you can start a handler on your phone and `npx tgcloud pull` it to your laptop later — nothing is tied to a single client. Running a handler even shows its **Console** output right in the chat, just like `npx tgcloud run`.

Projects and modules

A serverless project is an ordinary folder under version control. Everything the platform runs sits in one folder, `tgcloud/`; the rest of the project is yours — a mini-app front-end, config, docs.

There is no build step for the bot, no `node_modules` at runtime, and no server entry point.

Anatomy of a project

```
example_bot/
├─ tgcloud/             # the deployed part — nothing else is
│  ├─ handlers/         # update handlers — flat, one level only
│  │  ├─ message.js
│  │  └─ callback_query.js
│  ├─ endpoints/        # functions your Mini App calls — flat, one level only
│  │  └─ getProfile.js
│  ├─ lib/              # shared modules; subdirectories allowed
│  │  ├─ reply.js
│  │  └─ internal/util.js
│  └─ schema.js         # database schema — one file
└─ .tgcloud/            # CLI state, git-ignored
```

Only `.js` files inside `tgcloud/` — `schema.js`, `lib/`, `handlers/`, `endpoints/` — are **deployed**. Everything else — Markdown, config files, the `.tgcloud/` folder, anything outside `tgcloud/` — stays on your machine. Module names are relative to `tgcloud/`. The file `tgcloud/handlers/message.js` is the module `handlers/message`, and that is the name `npx tgcloud run` takes.

> **Upgrading from an older project?** Projects created before CLI 0.2.0 kept `schema.js`, `lib/` and `handlers/` at the project root. Run `npx tgcloud upgrade` once to move them into `tgcloud/`.

`schema.js` — your database. It declares tables as named exports using the schema DSL and lives in `tgcloud/` as a single file. It is deployed like any other module, but deploying it never changes the database — schema changes are applied separately with `npx tgcloud migrate`. See The database.

`lib/` — shared code, anything you want to reuse across handlers: pure helpers, database access layers, formatting, integrations with outside services. `lib/` is the only directory that may contain **subdirectories** (`lib/internal/util.js`, `lib/payments/stripe.js`), so you can organize a larger codebase however you like. Modules in `lib/` are never invoked directly by the platform; they exist to be imported by handlers and by each other.

`handlers/` — the entry points of your bot. Each file corresponds to one **Telegram update type**, and the platform routes each incoming update to the matching handler:

File	Handles
`handlers/message.js`	New incoming messages
`handlers/inline_query.js`	New incoming inline queries

File	Handles
`handlers/callback_query.js`	New incoming callback queries
…	*any other Bot API update type*

`handlers/` is **flat** — no subdirectories. A handler's `export default` is the function the platform calls (see Handlers).

An update type is handled **only if its handler file exists and is non-empty**. If there is no `handlers/<type>.js` — or the file is empty — updates of that type are ignored, and the platform runs nothing for them. So keep only the handlers you actually need: each one you add is another update type your bot wakes up to process, and leaving out the rest means Telegram doesn't spin up your code for updates you'd only discard anyway.

> To scaffold a new handler, run `npx tgcloud add handlers/<type>`.

`endpoints/` — the server side of your Mini App. Each file is a function the Mini App calls by name, so `endpoints/getProfile.js` answers `POST /api/getProfile`. The platform checks that the call comes from a Telegram user inside your Mini App before your code runs, and hands you that user. `endpoints/` is **flat**, like `handlers/`. See Endpoints.

`.tgcloud/` — machine-local state managed entirely by the CLI, including your saved credentials. It is git-ignored, and you should never read from or write to it by hand — use the CLI commands instead.

The module system

At runtime, a module can see exactly two things: **the platform SDK** and **the other modules in your project**. There are no npm packages, no filesystem, and no network except through the SDK's `fetch`.

Import your own modules the way you would in any JavaScript project — by **relative path, with the `.js` extension** — and the SDK by its name:

```
// tgcloud/handlers/message.js
import { users } from '../schema.js';              //  the schema module
import { addItem } from '../lib/cart.js';          //  a lib module
import { format } from '../lib/internal/fmt.js';  //  nested lib module
import { db, api, fetch } from 'sdk';              //  the platform SDK
```

```
// tgcloud/lib/cart.js
import { format } from './internal/fmt.js';        //  a neighbour
import { users } from '../schema.js';
```

A relative import must end in `.js` and can't lead outside `tgcloud/`:

```
import { users } from '../schema';       //  add the .js
import config from '../../config.js';    //  outside tgcloud/
import { z } from 'zod';                 //  no npm packages
```

Every module also has a **name** — its path inside `tgcloud/` without the extension, such as `schema`, `lib/cart` or `handlers/message`. That name is what `npx tgcloud run` and the CLI's output use, and a module can be imported by it as well (`from 'lib/cart'`, no extension). Prefer relative imports in your code. They are what your editor understands — jump-to-definition, autocomplete, rename.

What's available at runtime is exactly two things: **`sdk`** and its submodules (`sdk/db`, `sdk/api`, `sdk/fetch`) — the whole platform surface, see The SDK — and **your own modules** in `tgcloud/`. That's the complete list. If your code `import`s anything else, it won't resolve. This constraint is what keeps modules fast to load and safe to run.

Handlers

A handler is a module in `handlers/` whose default export the platform invokes when a matching update arrives.

```
// handlers/message.js
import { api } from 'sdk';

export default async function (message) {
  await api.sendMessage({
    chat_id: message.chat.id,
    text: `You said: ${message.text ?? '(no text)'}`,
  });
}
```

A handler receives the update's **payload** as its argument — the platform unwraps the Telegram Update for you. `handlers/message.js` gets the Message (i.e. `update.message`); `handlers/callback_query.js` gets the CallbackQuery; and so on. The handler's **second argument** is a per-invocation context object, `ctx`. It carries the raw `Update` as `ctx.update` — reach for it when you need something outside the payload, like `update_id`.

A handler can be `async` (usually is) and may return a value. It reaches the Bot API, the database, and outbound HTTP through the SDK.

You don't need a real update to test a handler. `npx tgcloud run` executes it on the platform with the payload you supply and your current local code:

```
npx tgcloud run handlers/message '{ chat: { id: 1 }, text: "hi" }'
```

The argument is the payload — the same object your handler receives — in JSON5. To supply the handler's `ctx` (its second argument), add `--ctx`, e.g. `--ctx '{ update: { update_id: 1 } }'`. This runs against your **local** files, so you can try changes before deploying them. See `run`.

Endpoints

An endpoint is a module in `endpoints/` that your **Mini App** calls — the place for anything the front-end can't do on its own, like reading and writing the database, calling the Bot API, or talking to outside services with your secrets.

```
// endpoints/getProfile.js
import { db, EndpointError } from 'sdk';
import { eq } from 'sdk/db';
import { profiles } from '../schema.js';

export default async function (input, ctx) {
  const user = ctx.initData.user;
  const profile = await db.select().from(profiles).where(eq(profiles.userId,
user.id)).get();
  if (!profile) throw new EndpointError('Profile not found', { userId: user.id
});
  return profile;
}
```

The first argument, `input`, is the JSON object the Mini App sent. The second, `ctx`, carries **`ctx.initData`** — the Mini App's init data (`user`, `chat`, `start_param`, `auth_date`, …), already **verified by the platform**. It has the same shape as `Telegram.WebApp.initDataUnsafe` on the client, but here you can trust it. A call without valid init data is rejected with `401` before your code runs, so `ctx.initData.user` really is the person who opened your Mini App. Endpoints can only be called this way — from your bot's Mini App, by a Telegram user — so there is no public API to protect and no login to build.

Whatever the endpoint returns is sent back as JSON. To refuse a call, `throw new EndpointError(description, parameters?)`. The Mini App gets the description and the `parameters` object as they are. If you need to branch on the kind of error in the front-end, put a code in `parameters` — `{ code: 'OUT_OF_STOCK' }`. Any other exception is reported as a generic internal error, and the real message goes only to your logs, so internals never leak to the client.

Calling an endpoint from the Mini App

The Mini App calls an endpoint by name with `Telegram.WebApp.Serverless.call` from the official Mini Apps SDK, `telegram-web-app.js` — the script your Mini App already includes. It sends the init data for you and calls back with an error or the endpoint's result:

```
Telegram.WebApp.Serverless.call('getProfile', { lang: 'en' }, function (err,
profile) {
  if (err) {
```

```
    // err.message — the description; err.parameters — what your EndpointError
carried
    if (err.type === 'ENDPOINT_ERROR') showMessage(err.message);
    return;
  }
  render(profile);
});
```

The input is optional and must be a plain object. Without it the call is `Serverless.call('getProfile', callback)`.

`err.type` is `'ENDPOINT_ERROR'` when your endpoint threw an `EndpointError`. Any other value means the platform refused the call or the endpoint failed unexpectedly, for example `'UNAUTHORIZED'` when the init data is missing or invalid.

Endpoint names are like JavaScript identifiers — letters, digits and `_`, not starting with a digit (`getProfile`, `place_order`). Scaffold one with `npx tgcloud add endpoints/<name>`, and try it without the Mini App with `run`, passing the init data yourself:

```
npx tgcloud run endpoints/getProfile '{}' --ctx '{ initData: { user: { id: 1,
first_name: "Ann" } } }'
```

What gets deployed

When you `npx tgcloud push`, the CLI gathers every `.js` file under `schema.js`, `lib/`, `handlers/`, and `endpoints/`, and sends that exact set as your project's module space — together with the Mini App's build folder, if `tgcloud.jsonc` names one (see below). Anything present in the cloud but absent from your project is removed, so the deployed state always mirrors your folder — deletions included. Nothing outside `tgcloud/` is deployed as code — the rest of the project root is yours. Inside `tgcloud/`, a `.js` file that isn't in one of those locations (`tgcloud/util.js`, `tgcloud/helpers/x.js`) is flagged, so it doesn't silently go undeployed. Markdown, dotfiles, and `.tgcloud/` are never deployed.

Mini App front-end

The project root is yours, and it is the natural place for your Mini App's front-end — built with React, Vue, Svelte or plain HTML, by Vite or any other bundler. Serverless hosts its **build output** next to your bot. One `push` deploys the code and the static files together, and the app is served at `https://app<app_id>.tgcloud.ai/` (`push` prints the exact address — use it as your Mini App's URL in @BotFather).

> **Reporting abuse.** Mini Apps on tgcloud.ai are published by independent developers. To report illegal content or abuse on an `app<app_id>.tgcloud.ai` subdomain, email abuse@telegram.org.

To start, create the front-end first, with whatever tool you like, then add Serverless on top of it:

```
npm create vite@latest example_app   # or create-vue, …
cd example_app
npm create @tgcloud/bot .
```

The project creator never overwrites your files. It adds the `tgcloud/` folder next to the front-end, recognizes the build folder, and adds an `npm run deploy` script that builds the app and deploys it together with the bot.

What gets hosted is set in **`tgcloud.jsonc`**, the project's config file at the project root:

```
{
  "$schema": "./node_modules/@tgcloud/cli/schema/tgcloud.json",
  "static": {
    "source": "dist",
    "spa": true,
    "immutable": ["/assets/*"],
    "headers":   [{ "path": "/*", "set": { "Referrer-Policy": "no-referrer" }
}],
    "redirects": [{ "from": "/old", "to": "/new", "status": 301 }]
  }
}
```

Only `"source"` is required. It is the build folder, relative to the project root. The other keys of `"static"` configure how it is served (below). `init` creates `tgcloud.jsonc` for you. When it recognizes the front-end, it also fills in `"source"`:

Front-end	**`init` sets `"source"` to**
Vite — React, Vue, Svelte, Solid, …	`dist`
Next.js, Nuxt, SvelteKit, React Router, TanStack Start, Angular	nothing — configure a static export first, then set `"source"` to its folder yourself

With any other setup, set `"source"` to your build folder yourself.

The `$schema` line gives your editor autocomplete and validation for every key. The file is JSON with comments and trailing commas allowed. If another tool of yours has to read it as plain JSON, name it `tgcloud.json` instead — then it must be strict JSON.

Build before you push. `push` uploads the folder exactly as it is on disk — except files and folders whose name starts with a dot, which are skipped — and never runs your build, so a plain `npx tgcloud push` after editing the front-end would deploy the previous build. Use `npm run deploy` (it runs your `build` script, then `tgcloud push`), or build yourself first.

The serving options:

- `spa` — serve `index.html` for paths that don't exist and have no file extension, so a client-side router works on reload. Paths with an extension still get `404`.
- `immutable` — patterns for content-hashed files (Vite's `/assets/*`) that browsers may cache for a year. Use it only for content-hashed files. Any other file matched here would stay stale in browsers for a year, and there is no way to recall it.
- `headers` — response headers for the paths a rule matches (an exact path, or a prefix ending in `/*`). A `null` value removes a header.
- `redirects` — redirect rules, where the first match wins. An existing file always beats a rule. `status` is `301` unless you set it.
- `"static": "dist"` — a shorthand for `{ "source": "dist" }` when you need nothing else.
- `"static": false` — remove the deployed site on the next push. Deleting the key instead just stops this project from managing static files, and whatever is deployed stays.

The site is served from the root of its own host, so your bundler's default `base` (`/`) is correct. Telegram Web opens Mini Apps in an iframe, so don't add `X-Frame-Options` or a restrictive `frame-ancestors` policy.

The same host answers your endpoints under `/api/` — that's where `Telegram.WebApp.Serverless.call('getProfile', …)` goes. Don't put files of your own under `/api/` in the build folder.

The database

Every bot gets its own database — an SQLite-backed store that persists between invocations and is available to every module through `db`. You describe your tables in `schema.js` with a small, typed DSL; you read and write them with a fluent query builder; and you evolve them with reviewed migrations.

> **Know Drizzle ORM?** Then you already know how to talk to the database here. The schema DSL and query builder follow Drizzle — the column builders, `select().from().where()`, the operators, `onConflictDoUpdate`, `.returning()`, and the `sql` tag all behave the way you'd expect, so reading and writing data is the familiar API you already use. You just import it from `sdk/db`, and a couple of platform specifics (most notably no foreign keys) are pointed out where they come up.

Declaring tables

Tables are **named exports** in `schema.js`. Calling `table()` builds a description at load time (it does not touch the database); the platform discovers the exported tables when you deploy `schema.js` and migrates the database to match.

```
import { table, integer, text, boolean, json, index, sql } from 'sdk/db';

export const users = table('users', {
  id:      integer('id').primaryKey({ autoIncrement: true }),
```

```
  tgId:    integer('tg_id').unique(),
  name:    text('name').notNull(),
  lang:    text('lang').default('en'),
  isAdmin: boolean('is_admin').default(false),
  prefs:   json('prefs'),
  created: integer('created_at', { mode: 'timestamp'
}).default(sql`(unixepoch())`),
}, (t) => ({
  createdIdx: index('idx_users_created').on(t.created),
}));
```

`table(name, columns, extras?)`:

- **`name`** is the SQL table name;
- **`columns`** is a map of JS property → column definition;
- **`extras`** is an optional callback `(t) => ({ … })` where `t` exposes the columns (`t.created` is a reference to that column) — declare indexes and table-level constraints here.

Column types

Factory	SQLite type	Notes
`text()`	TEXT	
`integer()`	INTEGER	
`real()`	REAL	alias `float()`
`numeric()`	NUMERIC	
`blob()`	BLOB	reads/writes `Uint8Array`
`boolean()`	INTEGER	stored as 0/1, read as `true` / `false`
`json()`	TEXT	auto `JSON.stringify` / `JSON.parse`

> The column name argument is optional — omit it and the JS key is used. The `mode` option controls how values convert between SQLite and JavaScript.

`mode`	**Stored as**	**JS value**
`boolean`	INTEGER 0/1	`boolean`
`json`	TEXT (JSON)	any object/array
`timestamp`	INTEGER (unix seconds)	`Date`
`timestamp_ms`	INTEGER (unix ms)	`Date`
`bytes`	BLOB	`Uint8Array`

`boolean()` and `json()` are shorthands for `integer(name, { mode: 'boolean' })` and `text(name, { mode: 'json' })`.

A `blob()` reads and writes a `Uint8Array` — the runtime has no Node `Buffer` (and a `Buffer` is a `Uint8Array` subclass, so this is the portable base type). The `mode` above governs only how a value is **encoded**, independent of the column's storage type: so `blob('col', { mode: 'json' })` is the BLOB counterpart of `json()` — the same JSON encoding, kept in a BLOB column rather than TEXT.

> **TLDR:** `blob()` uses `Uint8Array`. `mode` controls encoding, not storage, so `blob(..., { mode: 'json' })` stores JSON as a BLOB, while `json()` stores it as TEXT.

Column modifiers

Column modifiers chain onto a column.

```
integer('id').primaryKey({ autoIncrement: true })
text('name').notNull()
text('tg').unique()
text('lang').default('en')
integer('created_at', { mode: 'timestamp' }).default(sql`(unixepoch())`)
text('slug').generatedAlwaysAs(sql`lower(name)`, { mode: 'stored' })  // or
'virtual'
text('email').deprecated('replaced by login')   // marks the column for removal
```

> `.default()` on a `json()` column encodes the value for you; a `sql`…`` default is passed through verbatim. `.deprecated()` is terminal — see Migrations.

Indexes and constraints

Indexes and constraints are declared in the `extras` callback, where the columns are in scope.

```
table('t', { /* … */ }, (t) => ({
  uq:     unique('uq_email').on(t.email),
  chk:    check('chk_done', sql`${t.done} in (0, 1)`),
  idx:    index('idx_name').on(t.col),
  uidx:   uniqueIndex('uidx_email').on(t.email),
  lower:  index('idx_lower').on(sql`lower(${t.email})`),          // expression
index
  active: index('idx_active').on(t.userId).where(sql`done = 0`),  // partial
index
}));
```

Table-level modifiers chain after `table(...)`: `.strict()`, `.withoutRowid()`, `.deprecated('reason')`.

No foreign keys

The runtime runs with `PRAGMA foreign_keys` **off**. A declared foreign key would be silently inert — no cascades, no orphan protection — which is worse than having none at all, so the DSL makes it impossible. Namely, `.references()` and table-level `foreignKey()` **throw when declared**, and a schema that uses them will not deploy.

You should model relationships with plain columns (`userId: integer('user_id')`) and enforce **integrity** in your application code: insert parents before children, delete children before parents, handle errors properly and sweep orphans with a `LEFT JOIN … WHERE parent.id IS NULL` when you need to.

> The absence of foreign keys is a deliberate constraint, not an oversight. Account for it early on when planning your bot.

Querying

`db` is a fluent query builder. **Every query is asynchronous** — the terminal methods (`.all()`, `.get()`, `.values()`, `.run()`) return Promises, so **always** `await`.

```
import { db } from 'sdk';
import { eq, and, desc, asc, count, sql } from 'sdk/db';
import { users, todos } from '../schema.js';

await db.select().from(todos).all();                          // all rows
await db.select().from(todos).where(eq(todos.id, 1)).get();   // first row or
null
await db.select().from(todos).values();                       // rows as value
arrays

await db.select().from(todos)
  .where(and(eq(todos.userId, uid), eq(todos.done, false)))
  .orderBy(desc(todos.priority), asc(todos.id))
  .limit(10).offset(20)
  .all();

// custom projection: { alias: columnRef | sqlExpr | aggregate }
await db.select({ id: todos.id, title: todos.text, n: count() })
  .from(todos).groupBy(todos.userId).having(sql`count(*) > ${1}`).all();

// row count — a helper, not a builder terminal:
await db.$count(todos);                         // all rows
await db.$count(todos, eq(todos.done, false)); // with a filter
```

- Chainable: `.where()`, `.orderBy()`, `.limit()`, `.offset()`, `.groupBy()`, `.having()`, `.distinct()`.
- Terminals: `.all()`, `.get()`, `.values()`.
- Count rows with `db.$count(table, where?)` (or `count()` in a projection).

Insert, update, delete:

```
await db.insert(todos).values({ userId: 1, text: 'Buy milk' }).run();
await db.insert(todos).values([{ text: 'A' }, { text: 'B' }]).run();   // batch
await db.insert(todos).values({ text: 'X' }).returning().run();        //
RETURNING *

await db.insert(users).values({ tgId: 42, name: 'Ann' })
  .onConflictDoUpdate({ target: users.tgId, set: { name: 'Ann' } }).run();

await db.update(todos).set({ done: true }).where(eq(todos.id, 1)).run();
await db.delete(todos).where(eq(todos.id, 1)).run();
```

> Note that a batch insert is one statement, so it's capped by SQLite's variable limit (`rows × columns`); if you go over it, the insert errors out — chunk it yourself.

Operators are imported from `sdk/db`:

```
import {
  eq, ne, gt, gte, lt, lte,
  like, notLike,
  isNull, isNotNull, and, or, not,
  between, notBetween, inArray, notInArray,
  count, sum, avg, min, max,
  asc, desc,
} from 'sdk/db';
```

`.where(a, b)` with multiple arguments is the same as `and(a, b)`. A comparison's second argument is a value by default, but may be another column or `sql`…`` — e.g. `eq(a.x, b.y)`. Aggregates (`count/sum/avg/min/max`) are sql fragments for a `.select({ … })` projection.

Raw SQL — when the builder isn't enough, drop to raw SQL. The mode is chosen by method: `db.run` for writes, `db.all` for many rows, `db.get` for one row, `db.values` for rows as positional arrays.

```
await db.run('UPDATE todos SET done = 1 WHERE id = :id', { ':id': 5 });
// → { rowsAffected: 3, lastInsertRowid: 0, rows: [] }

await db.all(sql`SELECT * FROM todos WHERE done = ${false}`);
```

```
await db.get(sql`SELECT count(*) AS c FROM todos`);
await db.values(sql`SELECT id, text FROM todos`);   // → [[1, 'buy milk']]
```

A write resolves to a **run result** — `rowsAffected`, `lastInsertRowid`, and `rows`, which carry a `RETURNING` clause if the SQL had one. Reach for `db.values` when a query projects two columns of the same name (a join like `SELECT u.id, o.id`): `db.all` keys rows by column name, so the second `id` would overwrite the first.

The `sql`…`` tag turns `${value}` into a bound parameter and `${table.column}` into an identifier, and splices nested `sql` fragments. Use `sql.raw('…')` for a literal with no parameters.

> Raw queries are **not bound to a table**, so their rows come back without mode conversion — a boolean is `0` / `1`, a JSON column is a string, a timestamp is a number. Only the table-bound builder converts values.

Migrations

Your database changes as your bot grows. The platform keeps that safe by separating **deploying code** from **changing data**, and by classifying every schema change by how risky it is.

Deploying never touches the database. When you `npx tgcloud push` a changed `schema.js`, the platform records the new schema and tells you what the database *would* change — but applies nothing:

```
npx tgcloud push       # deploy schema.js; reports pending DB changes
npx tgcloud migrate    # review and apply them
```

`npx tgcloud migrate` computes the difference between your schema and the live database and walks you through it. You are always asked before anything is applied. This means a routine code deploy can never trigger a data migration by accident.

Each pending change carries a status that determines how `migrate` treats it:

Status	Meaning	In `migrate`
safe	Additive and non-blocking — a new table, column, or index	Applied together in one step, on confirmation
warning	Potentially destructive or slow — dropping something, or an index on a huge table	Presented one at a time, each confirmed separately
manual	Can't be done automatically — e.g. changing a column's type	Shown with guidance; you perform it by hand
undocumented	Exists in the database but not in your schema	Shown for awareness; not applied

Safe changes are quick and reversible in spirit, so they go through together. Each warning is a deliberate, individual confirmation — there is no “apply all” for destructive changes.

Manual changes come with a reason and a suggested action. `migrate` ends with a summary: how many changes were applied, skipped, awaiting a manual fix, or not in your schema.

> See `migrate` for the flags (`--dry-run`, `--safe`, `--yes`, `--local`).

Removing things

Deleting a table or column from `schema.js` does **not** drop it — that would make an accidental deletion catastrophic. To remove something, mark it deprecated:

```
// drop a column
text('email').deprecated('replaced by login')

// drop a whole table
export const oldSessions = table('old_sessions', { /* … */
}).deprecated('unused');
```

On the next `migrate`, the deprecated object shows up as a **warning**-status drop that you confirm individually. Once it's gone, remove the declaration.

Changing a column's type

Type changes are **manual** — SQLite can't always do them in place, and coercing existing values is a judgment call. `migrate` will show the change and its reasoning; perform it yourself with raw SQL (`db.run(...)`), typically by creating a new column or table, copying data, and swapping.

The SDK

At runtime a module has one library: `sdk`. It bundles the three things a bot backend needs — a database, the Telegram Bot API, and outbound HTTP — with nothing to install and no credentials to configure. The database (`db`) is covered in The database; this section covers `api`, `fetch`, and the `console` global.

```
import { db, api, fetch, InputFile, BotApiError, EndpointError } from 'sdk';
// the whole surface
// or from submodules:
import { table, integer, text, eq, sql } from 'sdk/db';
import { api } from 'sdk/api';
import { fetch } from 'sdk/fetch';
```

Import	What it is
`db`	Database — query builder and schema DSL → The database
`api`	Telegram Bot API — `api.sendMessage(...)` → below
`fetch`	Outbound HTTP → below
`InputFile`	A file (bytes + name) to upload — works with both `api` and `fetch` → Files
`EndpointError`	Refuse a Mini App's call with a message it can show → Endpoints

Import your own project modules by relative path with the `.js` extension (`from '../schema.js'`, `from '../lib/cart.js'`). See The module system.

The Bot API

`api` gives you the entire Telegram Bot API. Call any method as `api.<method>(params)`. Every current — and future — Bot API method works with **no SDK update** required.

```
import { api } from 'sdk';

const me = await api.getMe();                              // → the unwrapped
result
await api.sendMessage({ chat_id: id, text: 'Hello!' });
await api.editMessageText({ chat_id, message_id, text: 'Updated' });
await api.answerCallbackQuery({ callback_query_id, text: 'Done' });
```

The response envelope is unwrapped. The Bot API normally wraps results in `{ ok: true, result: … }`. `api` returns the `result` directly — `getMe()` resolves to the user object, not to a wrapper. Parameters use the Bot API's own snake_case names (`chat_id` , `message_id` , `reply_markup` , …).

Failures throw `BotApiError`. When the Bot API returns `{ ok: false }` , the call throws a `BotApiError` instead of returning a falsy value, so you can't accidentally ignore it. The error carries `.code` (the Bot API `error_code`), `.description` (the human-readable message), `.method` (which method failed), and `.parameters` (extra data such as `retry_after` on a 429, or `migrate_to_chat_id`). Catch it to handle an expected failure and rethrow the rest:

```
import { api, BotApiError } from 'sdk';

try {
  await api.deleteMessage({ chat_id, message_id });
} catch (e) {
  if (e instanceof BotApiError && e.code === 400) {
    // 400 = the message is already gone; that's fine here.
  } else {
    throw e;
```

```
  }
}
```

Files

Work with file **bytes**, not just `file_id`s — upload new files and download existing ones.

Upload — `InputFile`. Wrap raw bytes in an `InputFile` (a plain holder: bytes, a filename, and an optional MIME `type`) and pass it wherever a Bot API method expects a file:

```
import { api, InputFile } from 'sdk';

const doc = new InputFile(bytes, 'report.pdf', { type: 'application/pdf' });
await api.sendDocument({ chat_id, document: doc });

// files nested inside params work too — e.g. an album:
await api.sendMediaGroup({ chat_id, media: [
  { type: 'photo', media: new InputFile(a, 'a.jpg', { type: 'image/jpeg' }) },
  { type: 'photo', media: new InputFile(b, 'b.jpg', { type: 'image/jpeg' }) },
]});
```

The bytes are streamed to Telegram as a real multipart upload — the file isn't buffered whole into a single call to the platform — but Telegram's Bot API limits still apply: a bot can currently upload up to **50 MB** per file (and **10 MB** for a photo).

Download — by `file_id`. Two helpers pull a file's bytes back. The platform resolves the `file_id` for you (it runs `getFile` and fetches the content behind the scenes), so you never touch the bot token or a download URL:

```
const bytes = await api.getFileContent(file_id);          // → Uint8Array (the
whole file)

const { file, body } = await api.getFileStream(file_id);  // stream it instead
file.file_unique_id;                                      // the getFile info
for await (const chunk of body) { /* Uint8Array */ }      // or buffer it:
await stream.bytes()
```

> Telegram's Bot API caps `getFile` downloads at **20 MB**, so that's the ceiling for both helpers.

HTTP

`fetch` is a `fetch`-like client for calling the outside world — third-party APIs, webhooks, anything over HTTP.

```
import { fetch } from 'sdk';

const res = await fetch('https://api.example.com/users', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ name: 'Pavel' }),
});
if (!res.ok) throw new Error(res.statusText);
const data = await res.json();
```

The response mirrors the web platform: `res.status`, `res.statusText`, `res.ok` (true for 200–299), `res.url`, `res.headers` (`.get()`, `.has()`, `.keys()`, `.entries()`), and body readers `await res.json()` / `await res.text()`.

> You can also read the body incrementally as a **stream** — `for await (const chunk of res.body) { … }` — which is how you consume server-sent events or token-by-token output from **AI APIs**.

Body helpers set the matching `Content-Type` for you:

```
await fetch(url, { method: 'POST', body: fetch.body.json({ a: 1 }) }); //
application/json
await fetch(url, { method: 'POST', body: fetch.body.form({ a: 1 }) }); // x-
www-form-urlencoded
await fetch(url, { method: 'POST', body: fetch.body.text('hi') });     //
text/plain
```

Request bodies can also carry **files and raw bytes**: pass a `Uint8Array`, an `InputFile`, or a `FormData` (for `multipart/form-data`) as `body`, and the bytes are streamed out — so you can upload to a third-party API the same way you send files to Telegram:

```
import { fetch, FormData, InputFile } from 'sdk';

const form = new FormData();
form.append('file', new InputFile(bytes, 'photo.jpg', { type: 'image/jpeg' }));
form.append('caption', 'from my bot');
const res = await fetch('https://api.example.com/upload', { method: 'POST',
body: form });
```

It otherwise behaves like the standard `fetch` you already know, with **two constraints**:

- The **response** is read as text — `await res.json()` / `await res.text()`, or streamed chunk-by-chunk with `res.body`.

- The **total response is capped at 30 MB**. That cap covers the whole response — streaming with `res.body` lets you process a large body incrementally, but it does not raise the limit.

Logging

The standard global `console` is available — nothing to import, it's just there as in any JavaScript. Its output is captured and shown by `npx tgcloud run`, which makes it your primary debugging tool during development.

```
console.log('processing', { chatId: id });   // log / debug — plain
console.info('started');                     // info  — blue
console.warn('rate limited');                // warn  — yellow
console.error(err);                          // error — red, with a stack trace
```

Each line is tagged with the `[file:line]` it came from. `console.error` and `console.trace` append a full stack, while `console.warn` does not. When you `npx tgcloud run` a module, these lines are printed with a colored prefix per level, the time since the run started, and the origin — see `run`.

Command-line interface

`tgcloud` is the bridge between your project folder and the cloud. It scaffolds projects, shows you what changed, deploys, runs modules, and applies database migrations. It needs Node.js 18 or newer. Two ways to get it:

```
# Recommended — create a project with the CLI installed into it:
npm create @tgcloud/bot example_bot

# Or install the CLI globally and init an empty folder:
npm install -g @tgcloud/cli
tgcloud init
```

The npm package is `@tgcloud/cli`, and the command it installs is `tgcloud`. Every command works from any subfolder of your project.

Command	Purpose
`init`	Scaffold a new project in the current folder
`add`	Scaffold a new module (a handler, an endpoint or a lib module)
`login`	Link the project to a bot (saves the token)
`status`	Show what changed locally vs. the cloud
`diff`	Show the line-by-line changes

Command	Purpose
`push`	Deploy changed modules and the Mini App's build to the cloud
`migrate`	Apply schema changes to the database
`run`	Execute a module on the platform without deploying
`fetch`	Check what is deployed, without changing local files
`pull`	Bring local files in line with the cloud
`reset`	Discard local changes; restore from the cloud state
`webhook`	Inspect and re-sync the platform-managed webhook
`completion`	Print a shell completion script (bash/zsh/fish)
`upgrade`	Move a pre-0.2.0 project layout into `tgcloud/`

Authentication

A project is tied to one bot by its token, which has the form `app<id>:<secret>`. The `app<id>` part is public and may be printed; the secret never appears in logs or errors.

The token is resolved in this order:

1. **`TGCLOUD_TOKEN`** environment variable — for CI; never written to disk.
2. **`.tgcloud/credentials`** — written by `npx tgcloud login`.
3. Neither → an error pointing you at `npx tgcloud login`.

The CLI **never prompts for a token mid-command**. Logging in is always the explicit `login` step. If a saved token becomes invalid, the CLI removes it and asks you to `login` again.

`init`

```
npx tgcloud init
```

Scaffolds a new project in the current directory: the `tgcloud/` folder with `schema.js`, `lib/`, `handlers/` and a starter handler, plus `tgcloud.jsonc`, `AGENTS.md`, `docs/`, and the `.tgcloud/` state folder. Run it in the root of an existing front-end project too — it adds the `tgcloud/` folder and the docs, and never overwrites your files. If it recognizes the front-end, it also points `tgcloud.jsonc` at the build folder and adds a `deploy` script (build, then push) to your `package.json` — only if you don't have a `deploy` script already. See Mini App front-end.

`init` refuses to create a project **inside another project**. Re-running it in a project's own root is fine and just adds anything missing.

`add`

```
npx tgcloud add <target>
```

Scaffolds a single new module, wired up and ready to edit — a handler, an endpoint or a `lib/` module.

```
npx tgcloud add handlers/callback_query   # a new update handler
npx tgcloud add endpoints/getProfile      # a new endpoint for the Mini App
npx tgcloud add lib/cart                  # a new shared module
```

> Note that `add` never overwrites an existing file.

The `<target>` is the module's path (a trailing `.js` is optional). For `handlers/`, the name must be a Telegram update type. The platform advertises the valid set, so an invalid name is rejected up front. `handlers/` and `endpoints/` are flat, while `lib/` may be nested (`lib/payments/stripe`).

The module name is required. Giving **just the directory** is an error — but a helpful one: for `handlers/` it lists the update types you don't already have, so you can copy one.

```
$ npx tgcloud add handlers
Error: Specify a name, e.g. "handlers/callback_query". Available handlers/
types: callback_query, inline_query, chat_member, …
```

> `<Tab>` completion offers the same set — see `completion`.

The generated file has a live `export default`, so the handler is active as soon as you deploy — there's nothing to uncomment. Deploy the new module with `push`.

`login`

```
npx tgcloud login
```

Prompts for your CLI access token — from @BotFather → your bot → Serverless → CLI Access → Access token, a separate token from your bot's API token — validates it against the platform, and saves it to `.tgcloud/credentials`.

> `login` is the **only** command that asks for a token, and it requires an interactive terminal. In CI, set `TGCLOUD_TOKEN` instead.

`status`

```
npx tgcloud status
```

Shows, per file, what has changed between your working directory and the deployed copy: modified, new, deleted, unchanged. It works offline, comparing against the state of the cloud as of your last `push`, `pull` or `fetch`. It also warns about stray `.js` files inside `tgcloud/` that won't be deployed.

For a project with a Mini App, `status` also shows what changed in its build folder since the last deploy.

`diff`

```
npx tgcloud diff
```

Like `status`, but shows the actual line-by-line differences for changed modules. Also offline.

`push`

```
npx tgcloud push [files...]
```

Deploys your project to the cloud in one atomic batch.

With **no arguments**, it deploys the whole project, and the deployed state is made to mirror your folder exactly — modules you deleted locally are removed in the cloud.

With **file or directory arguments** (`npx tgcloud push tgcloud/handlers/message.js`, `npx tgcloud push handlers/`), it deploys only the changes in those files. A targeted push never deletes other modules. A target may be spelled as the file path (`tgcloud/handlers/message.js`), the path inside `tgcloud/` (`handlers/message.js`) or the module name (`handlers/message`) — they all mean the same module.

Its one option is `--force` — to skip the concurrency check and overwrite whatever is in the cloud. Only use it when you're sure (see Staying in sync).

If `tgcloud.jsonc` names a static folder, a full `push` deploys it in the same batch — only the files that changed are uploaded — and prints the site's address. Build first, because `push` never runs your build (`npm run deploy` does both). `npx tgcloud push dist/` deploys the static files alone, and a push that names only modules leaves them untouched.

After a deploy, if `schema.js` changed and the database is out of sync, `push` prints a summary of the pending changes and suggests `npx tgcloud migrate`. It never applies them itself.

`migrate`

```
npx tgcloud migrate
```

Applies your schema changes to the database. It computes the difference between `schema.js` and the live database, then walks you through it one step at a time with a running `[N/M]` counter:

- A brief summary of all pending changes.
- **Safe changes**, applied together in a single step on your confirmation.
- **Warnings** (drops, slow operations), one at a time, each confirmed separately.
- **Manual changes**, shown with a reason and suggested action, not applied automatically.
- **Undocumented objects** (in the database but not your schema), shown for awareness.

It ends with a summary: applied, skipped, awaiting manual fix, not in schema.

> See Migrations for the model.

Options: `--dry-run` (print everything, apply nothing), `--safe` (auto-apply safe changes, skip warnings), `--yes` (auto-apply safe changes and every warning, skip manual — use with care), `--local` (diff against your local `schema.js` instead of the deployed one). Without a flag, `migrate` requires a terminal and errors in a non-interactive environment rather than guessing.

`run`

```
npx tgcloud run <module> [args] [--ctx <json5>]
```

Executes a handler or an endpoint on the platform **without deploying**, using your current local files. This is the fast inner loop for testing logic.

- `<module>` — a bare name (searched under `handlers/` and `endpoints/`) or a path like `handlers/message`. A bare name that exists in both directories must be spelled with its directory.
- `[args]` — the payload passed to the handler, written in JSON5 so you can skip quoting keys. It's the update-type object your handler receives (e.g. a Message for `handlers/message`), or the `input` of an endpoint, and it must be an **object** — not an array or a bare string.
- `--ctx <json5>` — the module's context object (its second argument), also JSON5. Use it to supply what your code reads off `ctx` — the raw update for a handler, `--ctx '{ update: { update_id: 1 } }'`, or the init data for an endpoint, `--ctx '{ initData: { user: { id: 1 } } }'`. `run` doesn't verify it, since you're testing your logic, not the platform's checks.

```
npx tgcloud run handlers/message '{ chat: { id: 1 }, text: "hi" }'
```

The platform runs the module against the module space assembled from your **local** project (so locally-changed `lib/` code is used too) and returns the return value, anything logged with `console.*`, and the elapsed time. Read big arguments from a file with `npx tgcloud run handlers/message "$(cat message.json5)"`.

`fetch`

```
npx tgcloud fetch
```

Checks what is currently deployed, without touching your working files. After it, `status` and `diff` compare against the latest cloud state. Useful to look at a conflict before deciding how to resolve it.

`pull`

```
npx tgcloud pull
```

Brings your local project in line with the cloud by updating your working files to the deployed state.

`pull` never writes your Mini App's build folder.

`reset`

```
npx tgcloud reset
```

Discards your local changes and restores the modules from the last known cloud state. Use it to throw away an experiment. The Mini App's build folder is not touched.

`webhook`

```
npx tgcloud webhook
npx tgcloud webhook sync [--drop-pending]
```

Telegram delivers updates to your bot through a webhook, which the platform manages for you — you never point it anywhere by hand. `npx tgcloud webhook` shows its current state: the URL, the `allowed_updates` list, how many updates are pending, the last delivery error (if any), and whether it is **in sync** with your deployed handlers.

“In sync” means the webhook points at the platform and its `allowed_updates` match the handlers you've deployed — so Telegram delivers exactly the update types you handle, and nothing else. Deploying a new handler (or removing one) can leave the webhook out of sync until it's refreshed; `npx tgcloud status` flags this too.

`npx tgcloud webhook sync` fixes it — it re-points the webhook at the platform and rebuilds `allowed_updates` from your deployed handlers. Add `--drop-pending` to discard updates Telegram had already queued before the sync (otherwise they're delivered once the webhook is healthy again).

`completion`

> **Note:** tab-completion works only when a bare `tgcloud` is on your `PATH` — so install it globally (`npm install -g @tgcloud/cli`) or otherwise put the binary on your `PATH`. It can't hook into `npx`.

```
tgcloud completion <bash|zsh|fish>
```

Prints a shell completion script to stdout. Enable it once, then `<Tab>` completes commands, flags, module directories, the handler update-types you don't have yet, and your local runnable modules.

```
# bash — needs the bash-completion package:
echo 'eval "$(tgcloud completion bash)"' >> ~/.bashrc

# zsh — ensure `autoload -U compinit && compinit` runs in your ~/.zshrc:
echo 'eval "$(tgcloud completion zsh)"' >> ~/.zshrc

# fish:
tgcloud completion fish > ~/.config/fish/completions/tgcloud.fish
```

Restart your shell (or re-source the file) afterwards. Running `tgcloud completion` with no shell prints these instructions again.

`upgrade`

```
npx tgcloud upgrade [--dry-run]
```

One-time migration for projects created before CLI 0.2.0, which kept `schema.js`, `lib/` and `handlers/` at the project root. It moves them into `tgcloud/`. Your link to the bot is kept, and nothing needs to be redeployed. `--dry-run` prints the planned moves without touching anything. Other commands don't run on the old layout until you upgrade.

Staying in sync

Every project has a monotonically increasing **revision** in the cloud, bumped on each deploy. The CLI remembers the revision it last synced with and sends it on each `push`. If the cloud has moved

ahead — because another machine or teammate deployed — the push is **rejected** instead of silently overwriting their work, and the CLI offers three ways forward:

```
npx tgcloud fetch           # see what changed in the cloud, then re-check
npx tgcloud pull            # bring the latest into your working files
npx tgcloud push --force    # overwrite the cloud state (dangerous)
```

This optimistic-concurrency check is why you can share a bot across a team without a lockstep deploy process. Commands exit non-zero on failure — a rejected deploy, a failed migration, an authentication error, a module that threw during `run` — so they compose cleanly in scripts and CI pipelines.